



Chapter 4

PROCESS SYNCHRONIZATION (Chapter 4-Part 2)

Đinh Công Đoàn

1



Content

- Background
- The Critical-Section Problem
- Synchronization Hardware
- Semaphores
- Classical Problems of Synchronization
- Critical Regions
- Monitors

Đinh Công Đoàn

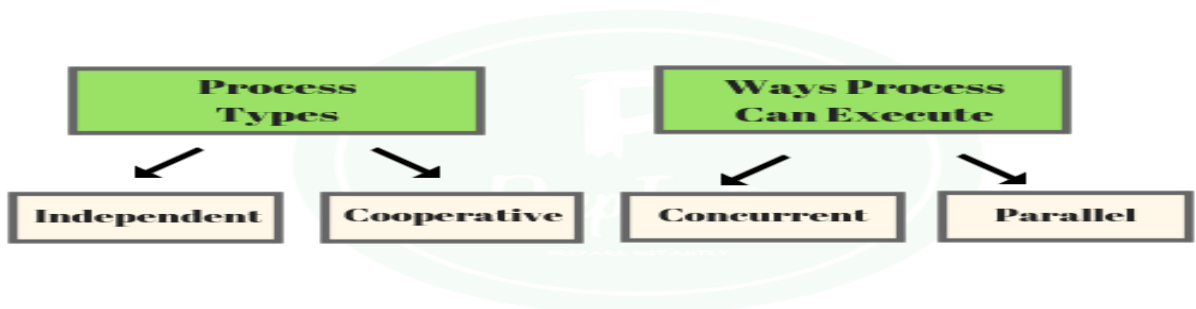
2

Introduction Process Synchronization in Operating System

- There are two ways any process can execute –
 - ✓ **In Concurrent Execution** – the CPU scheduler switches rapidly between processes. A process is stopped at any points and the processor is assigned to another instruction execution. Here, only one instruction is executed at a time.
 - ✓ **Parallel execution** – 2 or more instructions of different process execute simultaneously on different processing cores.



Process Synchronization Types in OS





Why is Process Synchronisation Important

- When several threads (or processes) share data, running in parallel on different cores, then changes made by one process may override changes made by another process running parallel. Resulting in inconsistent data. So, this requires processes to be synchronized, handling system resources and processes to avoid such situation is known as Process Synchronization.



Example 1: Classic Banking Example

- ✓ Consider your bank account has 5000\$.
- ✓ You try to withdraw 4000\$ using net banking and simultaneously try to withdraw via ATM too.
- ✓ For Net Banking at time $t = 0\text{ms}$ bank checks you have 5000\$ as balance and you're trying to withdraw 4000\$ which is lesser than your available balance. So, it lets you proceed further and at time $t = 1\text{ms}$ it connects you to server to transfer the amount
- ✓ Imagine, for ATM at time $t = 0.5\text{ms}$ bank checks your available balance which currently is 5000\$ and thus let's you enter ATM password and withdraw amount.
- ✓ At time $t = 1.5\text{ms}$ ATM dispenses the cash of 4000\$ and at time $t = 2$ net banking transfer is complete of 4000\$.
- Effect on the system
 - ✓ Now, due to concurrent access and processing time that computer takes in both ways you were able to withdraw 3000\$ more than your balance. In total 8000\$ were taken out and balance was just 5000\$.

Ví dụ 2 : Ứng dụng rút tiền

- ứng dụng kế toán, đa người dùng, đa nhiệm
 - ✓ Khi cần rút tiền trong tài khoản chung, khởi tạo 1 tiến trình : tiến trình rút tiền
 - ✓ Được phép rút tiền khi : số tiền cần rút < số tiền còn trong tài khoản
 - ✓ Giả sử có 2 tiến trình rút tiền P1 và P2 có thể hoạt động đồng thời
 - ✓ Biến :
 - tài khoản : số tiền còn trong tài khoản (tài nguyên găng)
 - tiền rút : số tiền cần rút
 - ✓ Khi cần rút tiền thực hiện đoạn mã sau

```

IF (Tài khoản - Tiền rút >= 0)      {kiểm tra tài khoản}
    Tài khoản := Tài khoản - Tiền rút    {thực hiện rút tiền}
Else
    Thông báo lỗi                      {không thể rút tiền}
EndIf;

```

- Nếu tại 1 thời điểm nào đó :
 - ✓ Trong tài khoản còn 800 (tài khoản = 800)
 - ✓ P1 cần rút 500 ngàn đồng (**Tiền rút = 500**)
 - ✓ P2 cần rút 400 ngàn đồng (**Tiền rút = 400**)
 - ✓ P1 và P2 đồng thời rút tiền.
- Về nguyên tắc không được phép do tổng số tiền rút > tài khoản
- Trong môi trường đa nhiệm, P1 và P2 vẫn có thể thực hiện thành công trong trường hợp sau :



- P1 được cấp CPU để thực hiện việc rút tiền: P1 kiểm tra tài khoản: **Tài khoản - Tiền rút = 800 - 500 = 300 > 0**, P1 ghi nhận điều này và chuẩn bị rút tiền
- Nhưng khi P1 chưa kịp rút tiền thì bị hệ điều hành thu hồi CPU, và hệ điều hành cấp CPU cho P2. P1 được chuyển sang trạng thái ready
- P2 nhận CPU, được chuyển sang trạng thái running, nó bắt đầu thực hiện rút tiền như sau: kiểm tra tài khoản: **Tài khoản - Tiền rút = 800 - 400 = 500 >= 0**, P2 ghi nhận điều này và thực hiện rút tiền:
- **Tài khoản = Tài khoản - Tiền rút = 800 - 400 = 400.**



How to solve this Situation

- To avoid such situations **process synchronisation is used**, so another concurrent process P2 is notified of existing concurrent process P1 and not allowed to go through as there is P1 process which is running and P2 execution is only allowed once P1 completes.
- Process Synchronization also prevents **race around condition**. It's the condition in which several processes access and manipulate the same data. In this condition, the outcome of the execution depends upon the particular order in which access takes place.



- On the basis of synchronization, processes are categorized as one of the following two types:
 - ✓ **Independent Process** : Execution of one process does not affects the execution of other processes.
 - ✓ **Cooperative Process** : Execution of one process affects the execution of other processes.



Problems

- To solve real world problems, one needs:
 - ✓ Apply multiprogramming to take advantage of CPU performance
 - ✓ For processes running concurrently to modularize or speed up computation → need mechanisms to exchange between processes → if there is no control mechanism when accessing shared data → data inconsistency (data inconsistency)
 - ✓ In order to maintain system data consistency there is a need for mechanisms to ensure the orderly execution of concurrent processes.



Learning Outcomes

- Understand concurrency is an issue in operating systems and multithreaded applications
- Know the concept of a *critical region*.
- Understand how mutual exclusion of critical regions can be used to solve concurrency issues
- Including how mutual exclusion can be implemented correctly and efficiently.
- Be able to identify and solve a *producer consumer bounded buffer* problem.
- Understand and apply standard synchronization primitives to solve synchronisation problems.

Đinh Công Đoàn

13



Background

- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.
- Shared-memory solution to bounded-buffer problem allows at most $n-1$ items in buffer at the same time. A solution, where all N buffers are used is not simple.
- Suppose that we modify the producer-consumer code by adding a variable counter, initialized to 0 and incremented each time a new item is added to the buffer

Đinh Công Đoàn

14

PROCESS SYNCHRONIZATION

The Producer Consumer Problem

A **producer** process "produces" information "consumed" by a **consumer** process. Here are the variables needed to define the problem:

```
#define BUFFER_SIZE 10
typedef struct {
    DATA          data;
} item;
item  buffer[BUFFER_SIZE];
int   in = 0;      // Location of next input to buffer
int   out = 0;     // Location of next removal from buffer
int   counter = 0; // Number of buffers currently full
```

Consider the code segments on the next page:

- Does it work?
- Are all buffers utilized?

6: Process Synchronization

15

PROCESS SYNCHRONIZATION

The Producer Consumer Problem

A **producer** process "produces" information "consumed" by a **consumer** process.

```
item  nextProduced;

while (TRUE) {
    while (counter ==
        BUFFER_SIZE);
    buffer[in] = nextProduced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

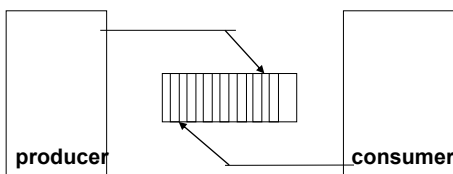
PRODUCER

```
#define BUFFER_SIZE 10
typedef struct {
    DATA          data;
} item;
item  buffer[BUFFER_SIZE];
int   in = 0;
int   out = 0;
int   counter = 0;
```

```
item  nextConsumed;

while (TRUE) {
    while (counter == 0);
    nextConsumed =
        buffer[out];
    out = (out + 1) %
        BUFFER_SIZE;
    counter--;
}
```

CONSUMER



6: Process Synchronization

16

PROCESS SYNCHRONIZATION

The Producer Consumer Problem

Note that **counter++;** ← this line is NOT what it seems!!
 is really --> register = counter
 register = register + 1
 counter = register

At a micro level, the following scenario could occur using this code:

T0;	Producer	Execute register1 = counter	register1 = 5
T1;	Producer	Execute register1 = register1 + 1	register1 = 6
T2;	Consumer	Execute register2 = counter	register2 = 5
T3;	Consumer	Execute register2 = register2 - 1	register2 = 4
T4;	Producer	Execute counter = register1	counter = 6
T5;	Consumer	Execute counter = register2	counter = 4

6: Process Synchronization

17

PROCESS SYNCHRONIZATION

Critical Sections

A section of code, common to n cooperating processes, in which the processes may be accessing common variables.

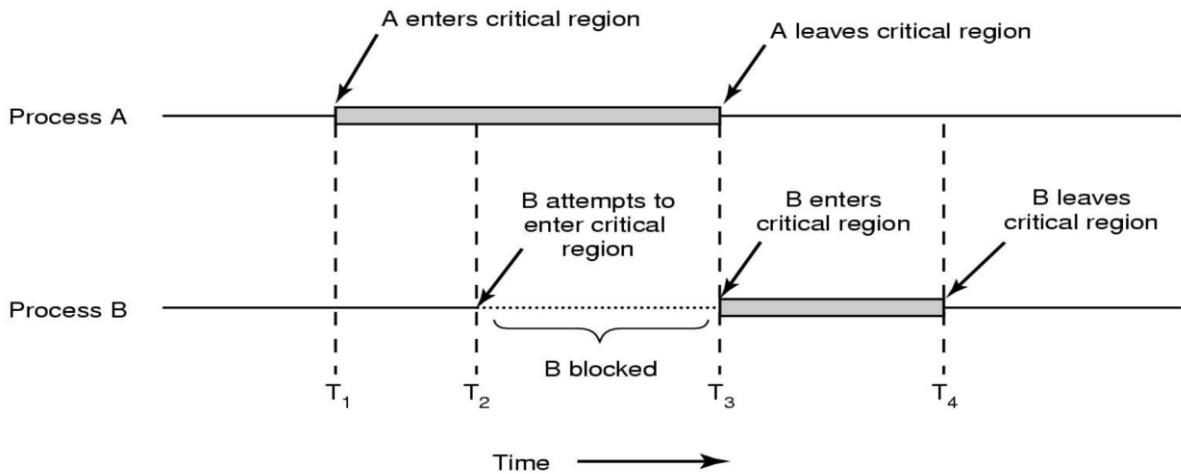
A Critical Section Environment contains:

Entry Section	Code requesting entry into the critical section.
Critical Section	Code in which only one process can execute at any one time.
Exit Section	The end of the critical section, releasing or allowing others in.
Remainder Section	Rest of the code AFTER the critical section.

6: Process Synchronization

18

Critical Regions



Mutual exclusion using critical regions

Đinh Công Đoàn

19

Race condition

- A race condition or race hazard is the behaviour of an electronic, software or other system where the output is dependent on the sequence or timing of other uncontrollable events.
- It becomes a bug when events do not happen in the intended order.
- The term originates with the idea of two signals racing each other to influence the output first

Đinh Công Đoàn

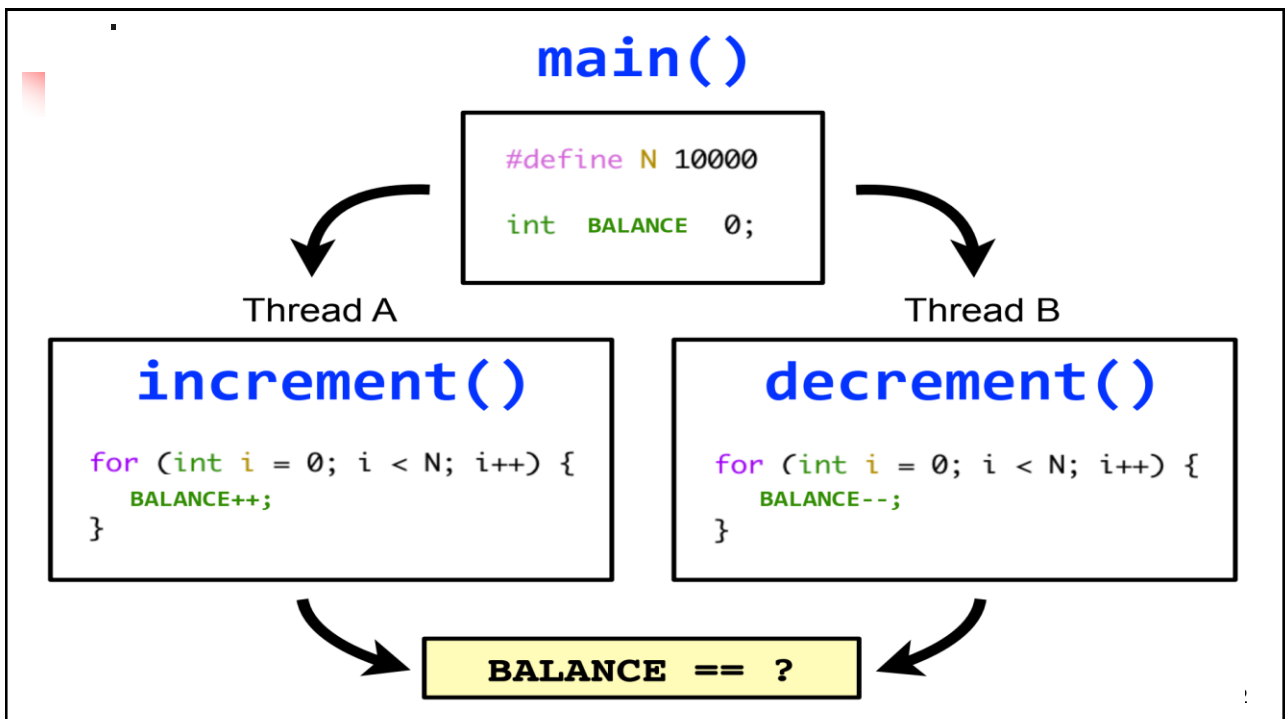
20

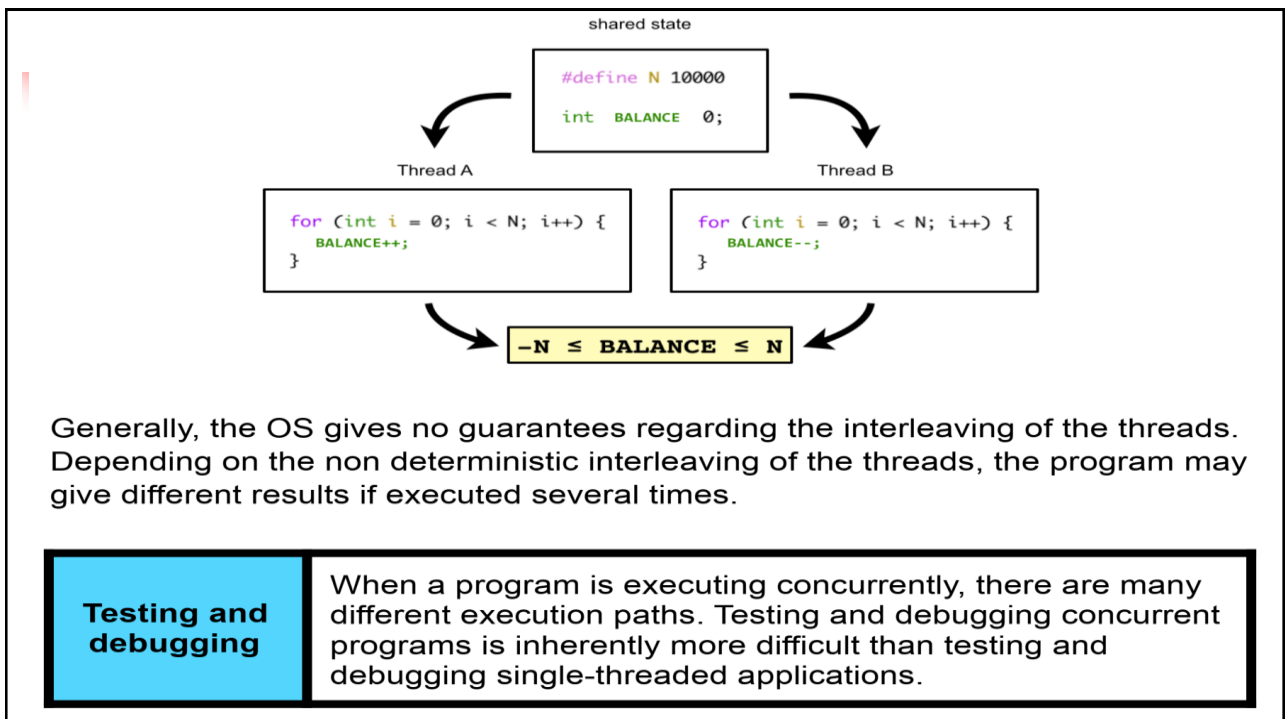
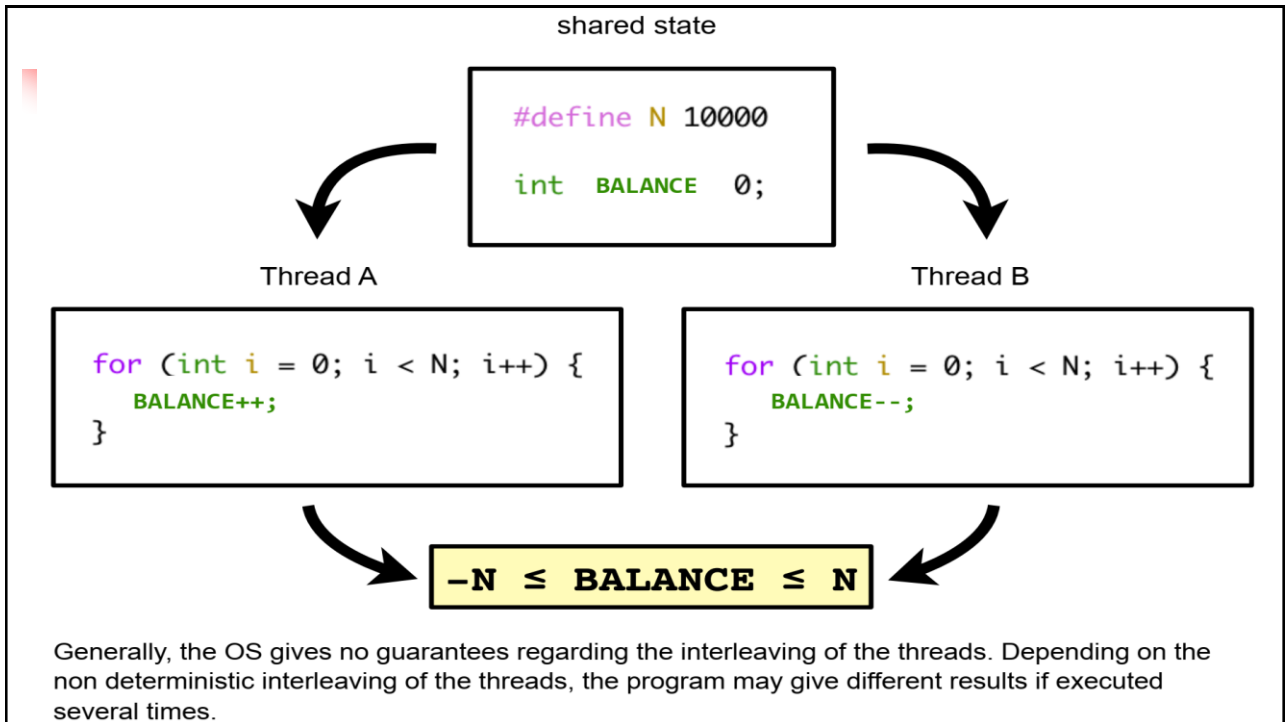
Data race

- A data race occurs when two instructions from different threads access the same memory location and:
 - ✓ at least one of these accesses is a write
 - ✓ and there is no synchronization that is mandating any particular order among these accesses

Đinh Công Đoàn

21







The Critical-Section Problem

- n processes all competing to use some shared data
- Each process has a code segment, called critical section, in which the shared data is accessed.
- Problem – ensure that when one process is executing in its critical section, no other process is allowed to execute in its critical section

- 
- N processes all competing to use shared data.
 - ✓ Structure of process P_i -- Each process has a code segment, called the critical section, in which the shared data is accessed.

repeat

entry section	/* enter critical section */
critical section	/* access shared variables */
exit section	/* leave critical section */
remainder section	/* do other work */

until false

- Problem
 - ✓ Ensure that when one process is executing in its critical section, no other process is allowed to execute in its critical section

PROCESS SYNCHRONIZATION

Critical Sections

The critical section must **ENFORCE ALL THREE** of the following rules:

Mutual Exclusion: No more than one process can execute in its critical section at one time.

Progress: If no one is in the critical section and someone wants in, then those processes not in their remainder section must be able to decide in a finite time who should go in.

Bounded Wait: All requesters must eventually be let into the critical section.

Properties of critical sections

- Assume that each process/thread executes at a nonzero speed. No assumption concerning relative speed of the N processes. In the following discussion the term **task** is used to mean a concurrent unit of execution such as a process or thread

Mutual Exclusion

If task T_i is executing in its critical section, then no other tasks can be executing in their critical sections.

Bounded Waiting

A bound must exist on the number of times that other tasks are allowed to enter their critical sections after a task has made a request to enter its critical section and before that request is granted.

Progress

If no task is executing in its critical section and there exist some task that wish to enter their critical section, then the selection of the task that will enter the critical section next cannot be postponed indefinitely.

More?

Deadlock

Fairness

Starvation

We will come back to this ...

Solution of the Critical Problem

- Software based solutions
- Hardware based solutions
- Operating system based solution
- Structure of Solution

✓ do {

entry section

Critical section

exit section

Reminder section

✓ } while (1);

Đinh Công Đoàn

29

Initial attempts to Solve Problem

- Only 2 processes, P0 and P1
- General structure of process P_i (other process P_j)

do {

entry section

(Primitive)

critical section

exit section

reminder section

} while (1);

- Processes may share some common variables to synchronize their actions

Đinh Công Đoàn

30

Algorithm 1 (Dekker 1)

- Shared variables:
 - int** **turn**;
 - initially **turn** = 0
 - turn** = **i** → P_i can enter its critical section
- Process P_i
 - do** {
 - while** (**turn** != **i**) ;
 - critical section
 - turn** = **j**;
 - remainder section
 - } while (1);**
- Satisfies mutual exclusion, but not progress

- **NOTE:** In all examples, i is the current process, j the "other" process. In these examples, envision the same code running on two processors at the same time.
- **Are the three Critical Section Requirements Met?**

Đinh Công Đoàn

31

Algorithm 1

- Here P_i and P_j share an integer variable *turn* initialized to 0. If $\text{turn} = i$, P_i can execute in its critical section.
- Mutual exclusion is assured but progress requirement is not since it requires strict alternation of processes in the execution of the critical section.
- E.g. if $\text{turn} = i$ and P_i does not want to enter the CS and P_j wants to enter the CS, then it can't.

Đinh Công Đoàn

32

Algorithm 2 (Dekker 2)

- P_i and P_j share an array: *boolean* $flag[2]$;
- $flag[0]$ and $flag[1]$ are initialized to false. If $flag[i]$ is true, then it indicates P_i is ready to enter its critical section.
- Shared variables: **boolean** $flag[2]$;
initially **flag** [0] = **flag** [1] = **false**.
flag [i] = **true** $\rightarrow$ P_i ready to enter its critical section

Process P_i

```

do {
    flag[i] := true;
    while (flag[j]) ;
    critical section
    flag [i] = false;
    remainder section
} while (1);

```

Đinh Công Đoàn

33

Algorithm 2

- Here too the progress requirement is not met even though mutual exclusion is satisfied.
- Need to remember that P_0 and P_1 are executing concurrently.
- Say,
 - ✓ T0: P_0 sets $flag[0]$ to true
 - ✓ T1: P_1 sets $flag[1]$ to true
 - ✓ Now both P_0 and P_1 are looping infinitely in their while loops.

Đinh Công Đoàn

34

Peterson's Solution

- Two process solution
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted.
- The two processes share two variables:
 - ✓ int **turn**;
 - ✓ boolean **flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section.
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i] = true** implies that process **P_i** is ready!

Đinh Công Đoàn

35

Algorithm 3 (Peterson)

- Combined shared variables of algorithms 1 and 2.
- Process P_i

```

do {
    flag [i] := true;
    turn = j;
    while (flag [j] and turn = j) ;
    critical section
    flag [i] = false;
    remainder section
} while (1);

```
- Meets all three requirements; solves the critical section problem for two processes.

Đinh Công Đoàn

36

Peterson's Solution contd..

- This algorithm meets all 3 conditions that a solution to CS problem must meet.
- Say P_i wants to enter its CS. It can be prevented only if $\text{flag}[j]$ is true and $\text{turn} = j$. If P_j is not ready to enter its CS, then $\text{flag}[j]$ is false and P_i will enter its CS.
- If P_j has set $\text{flag}[j]$ to true and is also executing in its while loop, then either $\text{turn} = i$ or $\text{turn} = j$ (depends on whether P_i or P_j accessed *turn* last) and so mutual exclusion is assured.
- If say P_j enters its CS then upon exiting it will set $\text{flag}[j]$ to false so P_i can now enter its CS.
- If P_j resets $\text{flag}[j]$ to true if it wants to enter CS again, then it will also $\text{turn} = i$. So P_i will enter its CS (progress is assured) after at most 1 entry by P_j (bounded waiting is assured).

Đinh Công Đoàn

37

Bakery Algorithm

- Critical section for n processes
 - ✓ Before entering its critical section, process receives a number. Holder of the smallest number enters critical section.
 - ✓ If processes P_i and P_j receive the same number,
 - if $i \leq j$, then P_i is served first; else P_j is served first.
 - ✓ The numbering scheme always generates numbers in increasing order of enumeration; i.e. 1,2,3,3,3,3,4,4,5,5

Đinh Công Đoàn

38

Bakery Algorithm (cont.)

■ Notation

- ✓ Lexicographic order(ticket#, process id#)
 - $(a,b) < (c,d)$ if $(a < c)$ or if $((a=c) \text{ and } (b < d))$
 - $\max(a_0, \dots, a_{n-1})$ is a number, k , such that $k \geq a_i$ for $i = 0, \dots, n-1$

■ Shared Data

var choosing: **array**[0.. $n-1$] **of** Boolean ;(initialized to false)
 number: **array**[0.. $n-1$] **of** integer ;(initialized to 0)

Đinh Công Đoàn

39

Bakery Algorithm (cont.)

repeat

```

  choosing[i] := true;
  number[i] := max(number[0], number[1], ..., number[n-1]) + 1;
  choosing[i] := false;
  for j := 0 to n-1
    do begin
      while choosing[j] do no-op;
      while number[j] < 0
        and (number[j], j) < (number[i], i) do no-op;
    end;
    critical section
    number[i] := 0;
    remainder section
  until false;
```

Đinh Công Đoàn

40

Bakery Algorithm

Process	Number
P0	3
P1	0
P2	7
P3	4
P4	8

Đinh Công Đoàn

41

Bakery Algorithm

P0	P2	P3	P4
$(3,0) < (3,0)$	$(3,0) < (7,2)$	$(3,0) < (4,3)$	$(3,0) < (8,4)$
Number[1] = 0	Number[1] = 0	Number[1] = 0	Number[1] = 0
$(7,2) < (3,0)$	$(7,2) < (7,2)$	$(7,2) < (4,3)$	$(7,2) < (8,4)$
$(4,3) < (3,0)$	$(4,3) < (7,2)$	$(4,3) < (4,3)$	$(4,3) < (8,4)$
$(8,4) < (3,0)$	$(8,4) < (7,2)$	$(8,4) < (4,3)$	$(8,4) < (8,4)$

Đinh Công Đoàn

42



Bakery Algorithm

- P1 not interested to get into its critical section → number[1] is 0
- P2, P3, and P4 wait for P0
- P0 gets into its CS, get out, and sets its number to 0
- P3 get into its CS and P2 and P4 wait for it to get out of its CS
- P2 gets into its CS and P4 waits for it to get out
- P4 gets into its CS
- Sequence of execution of processes: <P0, P3, P2, P4>



Bakery Algorithm

- Meets all three requirements:
 - ✓ Mutual Exclusion: $(\text{number}[j], j) < (\text{number}[i], i)$ cannot be true for both P_i and P_j
 - ✓ Bounded-waiting: At most one entry by each process ($n-1$ processes) and then a requesting process enters its critical section (First-Come-First-Serve)
 - ✓ Progress: Decision takes complete execution of the 'for loop' by one process. No process in its 'Remainder Section' (with its number set to 0) participates in the decision making

Supporting Synchronization

<i>Programs</i>	<i>Shared Programs</i>
<i>Higher-level API</i>	<i>Locks Semaphores Monitors Send/Receive CRegions</i>
<i>Hardware</i>	<i>Load/Store Disable Ints Test&Set Comp&Swap</i>

- We are going to implement various synchronization primitives using atomic operations
 - ✓ Everything is pretty painful if only atomic primitives are load and store
 - ✓ Need to provide inherent support for synchronization at the hardware level
 - ✓ Need to provide primitives useful at software/user level

Đinh Công Đoàn

45

Synchronization Hardware

- Many systems provide hardware support for critical section code.
- Uniprocessors – could disable interrupts
 - ✓ Currently running code would execute without preemption.
 - ✓ Generally too inefficient on multiprocessor systems.
 - ✓ Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - ✓ Atomic = non-interruptable
 - ✓ Either test memory word and set value often called a **TAS** (Test And Set) instruction.
 - ✓ Or **SWAP** contents of two memory words.

Đinh Công Đoàn

46



TestAndSet

- Here we use C to define the semantics of the TestAndSet (TAS) instruction. Optimally, the TAS instruction is supported directly by the CPU

```
boolean TestAndSet (boolean *target) {  
    boolean rv = *target;  
    *target = TRUE;  
    return rv;  
}
```

Đinh Công Đoàn

47

- 
- **TestAndSet** set the value at the target address to True and return the old value at address target.
 - The **TestAndSet** instruction must be **atomic**. If two such instructions are executed simultaneously on the same target address (each on a different CPU or core), they will be executed sequentially in some arbitrary order.

Đinh Công Đoàn

48

Mutual Exclusion with TestandSet

- Shared data: var lock: boolean (initially false)

- **Process P_i**

repeat

while Test-and-Set (lock) **do** no-op;

critical section

lock := false;

remainder section

until false;

- once we're allowed to enter the CS, the lock is **atomically** set to true blocking others from entering

Đinh Công Đoàn

49

Bounded Waiting Mutual Exclusion with Test-and-Set

- **var** j : 0..n-1;
key : boolean;
repeat
waiting [i] := true; key := true;
while waiting[i] **and** key **do** key := Test-and-Set(lock);
waiting [i] := false;
critical section
j := i+1 mod n;
while (j <> i) **and** (**not** waiting[j]) **do** j := j + 1 mod n;
if j = i **then** lock := false;
else waiting[j] := false;
remainder section
until false;

Đinh Công Đoàn

50



Solution with TestandSet

- Is the TSL-based solution good? No
- Mutual Exclusion: Satisfied
- Progress: Satisfied
- Bounded Waiting: Not satisfied



Synchronization Hardware - Swap

- Atomically swap two variables.
- **Swap** swaps the content of two memory locations a and b

```
void Swap(boolean &a, boolean &b)
{
    boolean temp = a;
    a = b;
    b = temp;
}
```



Mutual Exclusion with Swap

- Shared data (initialized to **false**):
 - boolean lock;**
 - boolean waiting[n];**
- Each process has a local **boolean** variable **key** and 'key' is set to false
- **Process P_i**
 - do {**
 - key = true;**
 - while (key == true)**
 - Swap(lock, key);**
 - critical section
 - lock = false;**
 - remainder section
 - }**

Đinh Công Đoàn

53



Solution with swap

- Using **Swap**, once we're allowed to enter the CS, the lock is **atomically** set to true blocking others from entering.
- Is the swap-based solution good? No
- Mutual Exclusion: Satisfied
- Progress: Satisfied
- Bounded Waiting: Not satisfied

Đinh Công Đoàn

54



Semaphores

- The hardware solutions to the critical section problem using Swap and TestAndSet are complicated for programmers to use.
- To overcome this difficulty, we can use a synchronization tool/abstraction called a semaphore
- Semaphores can be used to implement various types of synchronization.
 - ✓ **Counting semaphore** – integer value can range over an unrestricted domain.
 - ✓ **Binary semaphore** – integer value can range only between 0 and 1; can be simpler to implement.
 - Also known as **mutex locks** (may include priority inheritance)

Đinh Công Đoàn

55



Semaphore

A variable that has an integer value upon which only three operations are defined:

- There is no way to inspect or manipulate semaphores other than these three operations

- 1) May be initialized to a nonnegative integer value
- 2) The semWait operation decrements the value
- 3) The semSignal operation increments the value

Consequences

There is no way to know before a process decrements a semaphore whether it will block or not

There is no way to know which process will continue immediately on a uniprocessor system when two processes are running concurrently

You don't know whether another process is waiting so the number of unblocked processes may be zero or one

Figure 5.3 A Definition of Semaphore Primitives

```
struct semaphore {
    int count;
    queueType queue;
};
void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue */;
        /* block this process */;
    }
}
void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}
```


Figure 5.4 A Definition of Binary Semaphore Primitives

```

struct binary_semaphore {
    enum {zero, one} value;
    queueType queue;
};
void semWaitB(binary_semaphore s)
{
    if (s.value == one)
        s.value = zero;
    else {
        /* place this process in s.queue */;
        /* block this process */;
    }
}
void semSignalB(semaphore s)
{
    if (s.queue is empty())
        s.value = one;
    else {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
}

```



Strong/Weak Semaphores

☺ A queue is used to hold processes waiting on the semaphore

Strong Semaphores

- the process that has been blocked the longest is released from the queue first (FIFO)

Weak Semaphores

- the order in which processes are removed from the queue is not specified

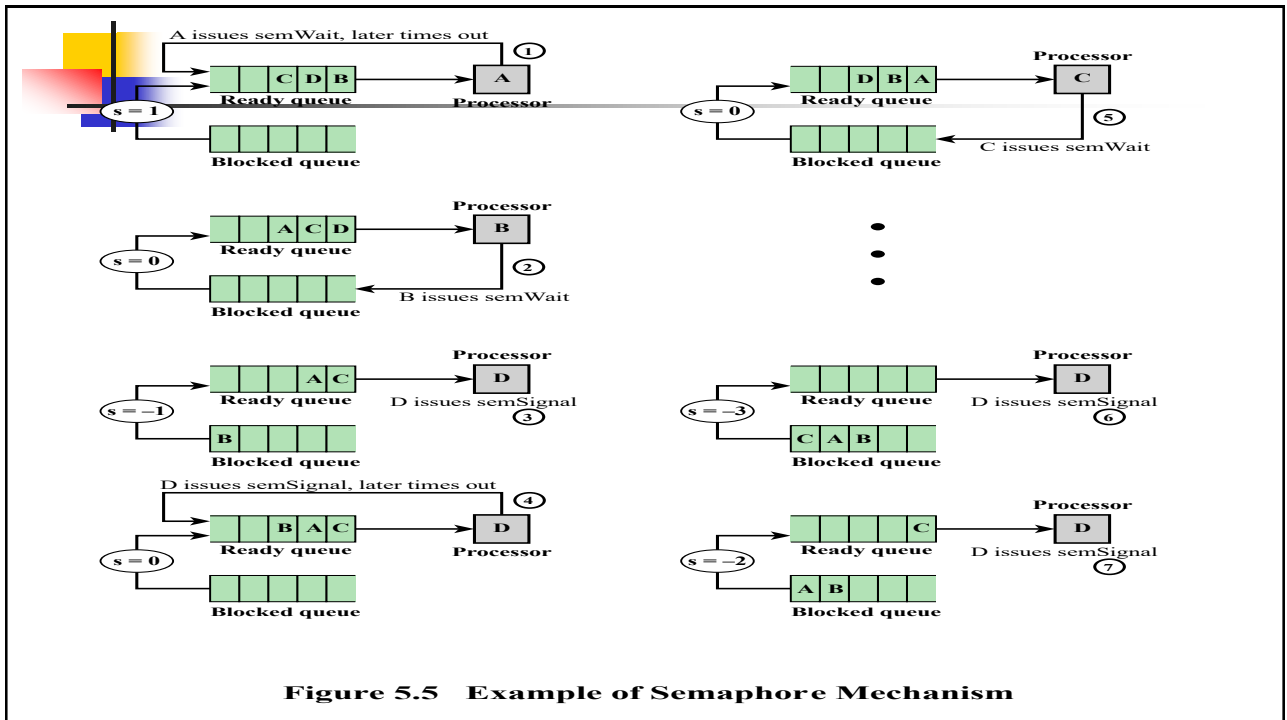


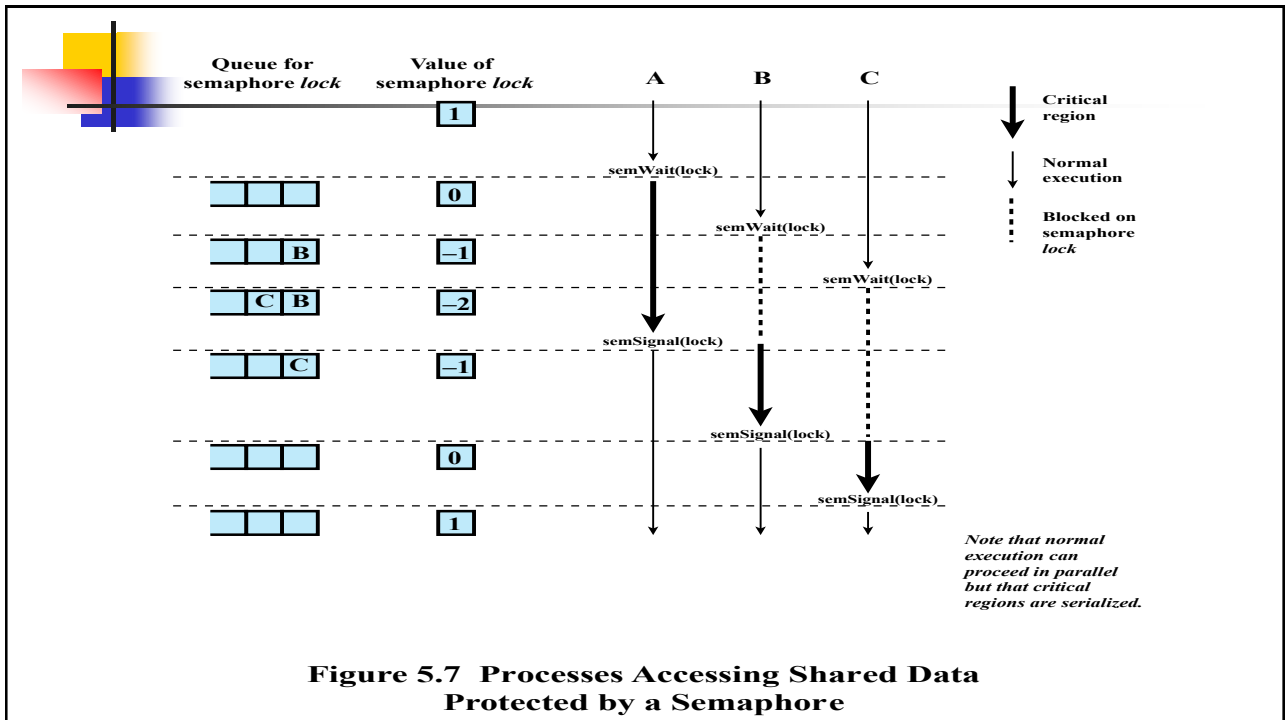
Figure 5.5 Example of Semaphore Mechanism

Figure 5.6 Mutual Exclusion Using Semaphores

```

/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int i)
{
    while (true) {
        semWait(s);
        /* critical section */;
        semSignal(s);
        /* remainder */;
    }
}
void main()
{
    parbegin (P(1), P(2), . . . , P(n));
}

```



Producer/Consumer Problem

General Statement:

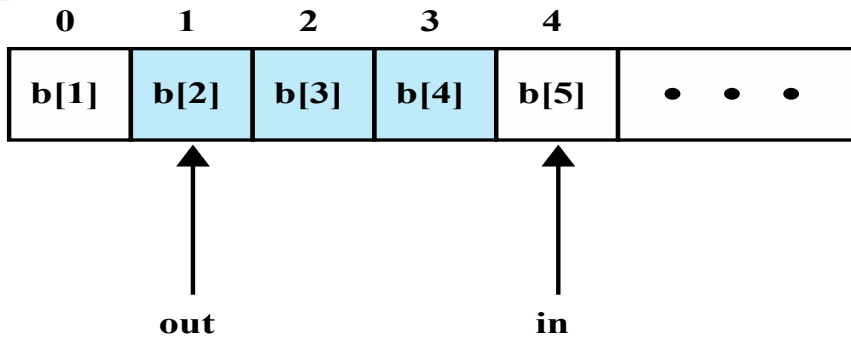
one or more producers are generating data and placing these in a buffer

a single consumer is taking items out of the buffer one at a time

only one producer or consumer may access the buffer at any one time

The Problem:

ensure that the producer can't add data into full buffer and consumer can't remove data from an empty buffer



Note: shaded area indicates portion of buffer that is occupied

Figure 5.8 Infinite Buffer for the Producer/Consumer Problem

```

/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
    while (true) {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}
void consumer()
{
    semWaitB(delay);
    while (true) {
        semWaitB(s);
        take();
        n--;
        semSignalB(s);
        consume();
        if (n==0) semWaitB(delay);
    }
}
void main()
{
    n = 0;
    parbegin (producer, consumer);
}

```

Figure 5.9
An Incorrect Solution
to the Infinite-Buffer Producer/Consumer
Problem Using Binary Semaphores

Table 5.4 Possible Scenario for the Program of Figure 5.9

	Producer	Consumer	s	n	Delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	if (n==1) (semSignalB(delay))		0	1	1
5	semSignalB(s)		1	1	1
6		semWaitB(delay)	1	1	0
7		semWaitB(s)	0	1	0
8		n--	0	0	0
9		semSignalB(s)	1	0	0
10	semWaitB(s)		0	0	0
11	n++		0	1	0
12	if (n==1) (semSignalB(delay))		0	1	1
13	semSignalB(s)		1	1	1
14		if (n==0) (semWaitB(delay))	1	1	1
15		semWaitB(s)	0	1	1
16		n--	0	0	1
17		semSignalB(s)	1	0	1
18		if (n==0) (semWaitB(delay))	1	0	0
19		semWaitB(s)	0	0	0
20		n--	0	-1	0
21		semSignalB(s)	1	-1	0

Note: White areas represent the critical section controlled by semaphore s.

Figure 5.10
A Correct Solution to the Infinite-Buffer
Producer/Consumer Problem Using Binary
Semaphores

```

/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
    while (true) {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}
void consumer()
{
    int m; /* a local variable */
    semWaitB(delay);
    while (true) {
        semWaitB(s);
        take();
        n--;
        m = n;
        semSignalB(s);
        consume();
        if (m==0) semWaitB(delay);
    }
}
void main()
{
    n = 0;
    parbegin (producer, consumer);
}

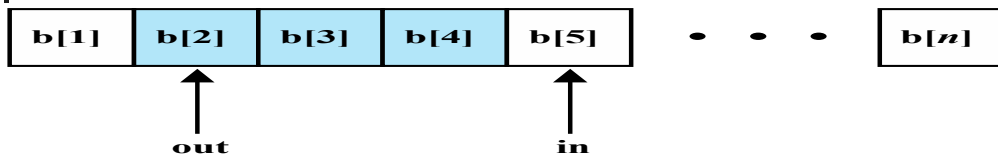
```

Figure 5.11
A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores

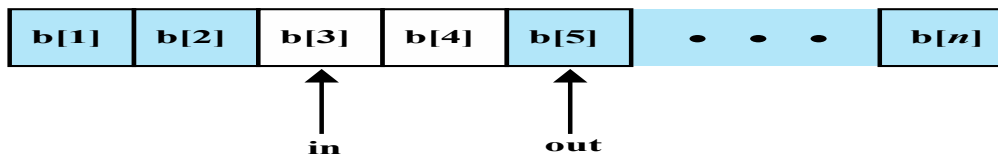
```

/* program producerconsumer */
semaphore n = 0, s = 1;
void producer()
{
    while (true) {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}

```



(a)



(b)

Figure 5.12 Finite Circular Buffer for the Producer/Consumer Problem

Figure 5.13
A Solution to the Bounded-Buffer
Producer/Consumer Problem Using Semaphores

```

/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n = 0, e = sizeofbuffer;
void producer()
{
    while (true) {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}

```

Implementation of Semaphores

- Imperative that the `semWait` and `semSignal` operations be implemented as atomic primitives
- Can be implemented in hardware or firmware
- Software schemes such as Dekker's or Peterson's algorithms can be used
- Use one of the hardware-supported schemes for mutual exclusion

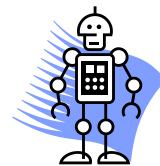


Figure 5.14 Two Possible Implementations of Semaphores

```
semWait(s)
{
    while (compare_and_swap(s.flag, 0, 1) == 1)
        /* do nothing */;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue*/;
        /* block this process (must also set s.flag to 0)
    */;
    }
    s.flag = 0;
}

semSignal(s)
{
    while (compare_and_swap(s.flag, 0, 1) == 1)
        /* do nothing */;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
    s.flag = 0;
}
```

(a) Compare and Swap Instruction

```
semWait(s)
{
    inhibit interrupts;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue*/;
        /* block this process and allow interrupts*/;
    }
    else
        allow interrupts;
}

semSignal(s)
{
    inhibit interrupts;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue*/;
        /* place process P on ready list*/;
    }
    allow interrupts;
}
```

(b) Interrupts

Monitors

- Programming language construct that provides equivalent functionality to that of semaphores and is easier to control
- Implemented in a number of programming languages
 - including Concurrent Pascal, Pascal-Plus, Modula-2, Modula-3, and Java
 - Has also been implemented as a program library
 - Software module consisting of one or more procedures, an initialization sequence, and local data

Monitor Characteristics

Local data variables are accessible only by the monitor's procedures and not by any external procedure



Process enters monitor by invoking one of its procedures



Only one process may be executing in the monitor at a time

Synchronization

- Achieved by the use of **condition variables** that are contained within the monitor and accessible only within the monitor
 - Condition variables are operated on by two functions:
 - cwait(c): suspend execution of the calling process on condition c
 - csignal(c): resume execution of some process blocked after a cwait on the same condition



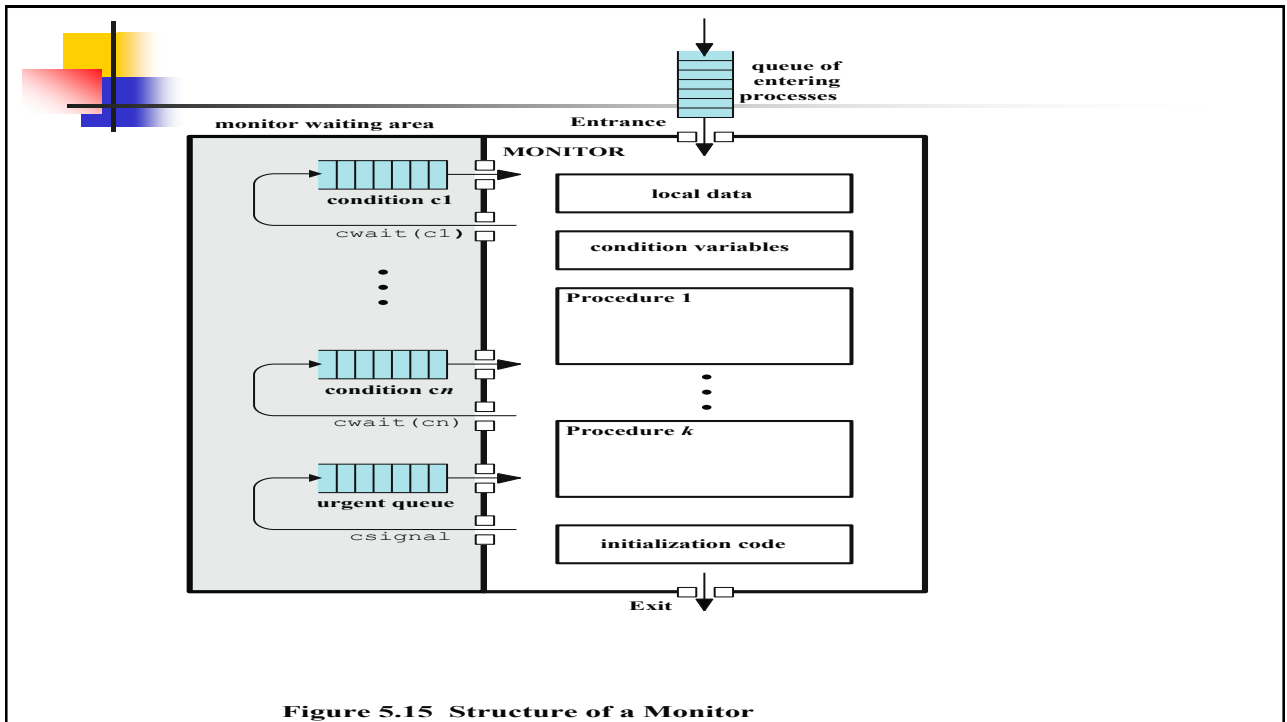


Figure 5.15 Structure of a Monitor

Figure 5.16
A Solution to the Bounded-Buffer
Producer/Consumer Problem Using a
Monitor

```

/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];
int nextin, nextout;
int count;
cond notfull, notempty;

/* space for N items */
/* buffer pointers */
/* number of items in buffer */
/* condition variables for synchronization */

void append (char x)
{
    if (count == N) cwait(notfull);    /* buffer is full; avoid overflow */
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;
    /* one more item in buffer */
    csignal(notempty);                /* resume any waiting consumer */
}

void take (char x)
{
    if (count == 0) cwait(notempty);   /* buffer is empty; avoid underflow */
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;
    csignal(notfull);                 /* resume any waiting producer */
}

/* monitor body */
/* buffer initially empty */
{
    nextin = 0; nextout = 0; count = 0;
}

void producer()
{
    char x;
    while (true) {
        produce(x);
        append(x);
    }
}

void consumer()
{
    char x;
    while (true) {
        take(x);
        consume(x);
    }
}

void main()
{
    parbegin (producer, consumer);
}

```



```

void append (char x)
{
    while(count == N) cwait(notfull); /* buffer is full; avoid overflow */
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;
    cnotify(notempty); /* one more item in buffer */
    /* notify any waiting consumer */
}

void take (char x)
{
    while(count == 0) cwait(notempty); /* buffer is empty; avoid underflow */
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;
    cnotify(notfull); /* one fewer item in buffer */
    /* notify any waiting producer */
}

```

Figure 5.17 Bounded Buffer Monitor Code for Mesa Monitor



Message Passing

- When processes interact with one another two fundamental requirements must be satisfied:

synchronization

- to enforce mutual exclusion

communication

- to exchange information

- Message Passing is one approach to providing both of these functions
 - works with distributed systems *and* shared memory multiprocessor and uniprocessor systems

Message Passing



- The actual function is normally provided in the form of a pair of primitives:

send (destination, message)

receive (source, message)

- A process sends information in the form of a *message* to another process designated by a *destination*
- A process receives information by executing the *receive* primitive, indicating the *source* and the *message*

Synchronization

Send
 blocking
 nonblocking
 Receive
 blocking
 nonblocking
 test for arrival

Addressing

Direct
 send
 receive
 explicit
 implicit
 Indirect
 static
 dynamic
 ownership

Format

Content
 Length
 fixed
 variable

Queueing Discipline

FIFO
 Priority

Table 5.5

Design Characteristics of Message Systems for Interprocess Communication and Synchronization

Synchronization

Communication of a message between two processes implies synchronization between the two

the receiver cannot receive a message until it has been sent by another process

When a receive primitive is executed in a process there are two possibilities:

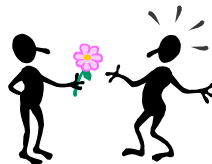
if there is no waiting message the process is blocked until a message arrives or the process continues to execute, abandoning the attempt to receive

if a message has previously been sent the message is received and execution continues



Blocking Send, Blocking Receive

- Both sender and receiver are blocked until the message is delivered
- Sometimes referred to as a *rendezvous*
- Allows for tight synchronization between processes



Nonblocking Send

Nonblocking send, blocking receive

- sender continues on but receiver is blocked until the requested message arrives
- most useful combination
- sends one or more messages to a variety of destinations as quickly as possible
- example -- a service process that exists to provide a service or resource to other processes

Nonblocking send, nonblocking receive

- neither party is required to wait



Addressing

- ★ Schemes for specifying processes in send and receive primitives fall into two categories:

Direct
addressing

Indirect
addressing



Direct Addressing



- Send primitive includes a specific identifier of the destination process
- Receive primitive can be handled in one of two ways:
 - require that the process explicitly designate a sending process
 - effective for cooperating concurrent processes
 - implicit addressing
 - source parameter of the receive primitive possesses a value returned when the receive operation has been performed

Indirect Addressing



Messages are sent to a shared data structure consisting of queues that can temporarily hold messages



Queues are referred to as *mailboxes*



Allows for greater flexibility in the use of messages



One process sends a message to the mailbox and the other process picks up the message from the mailbox

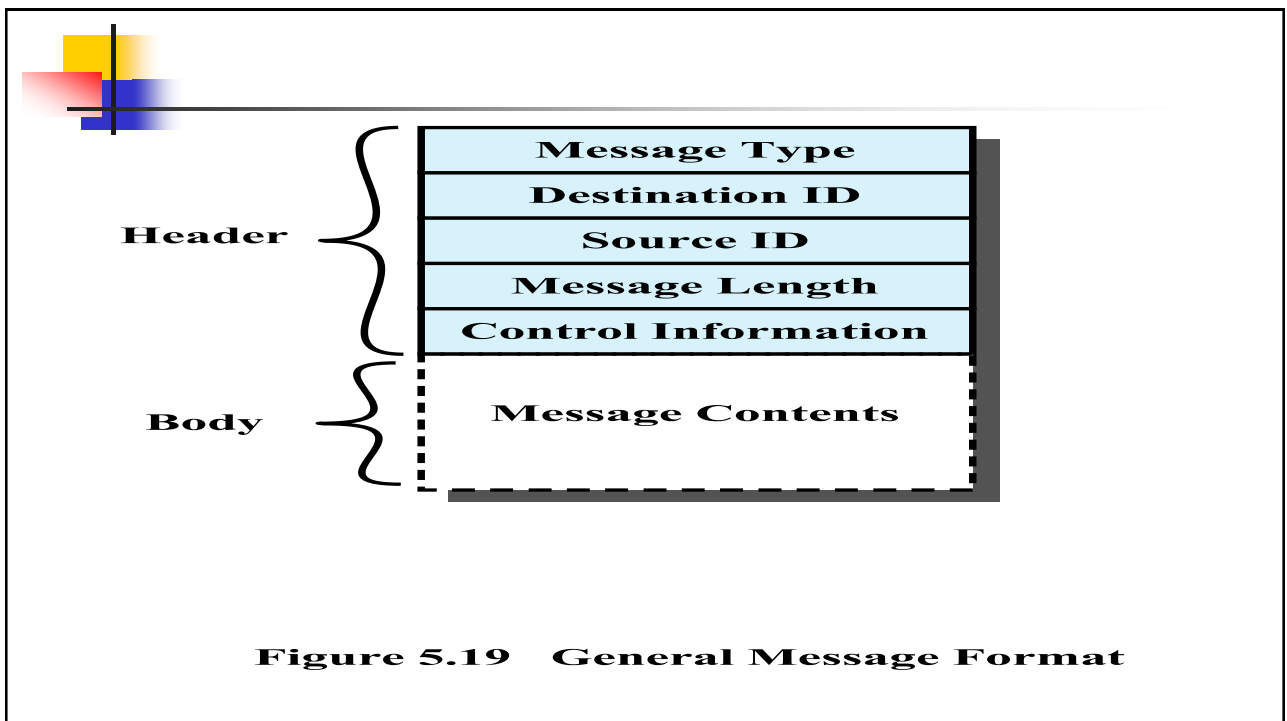
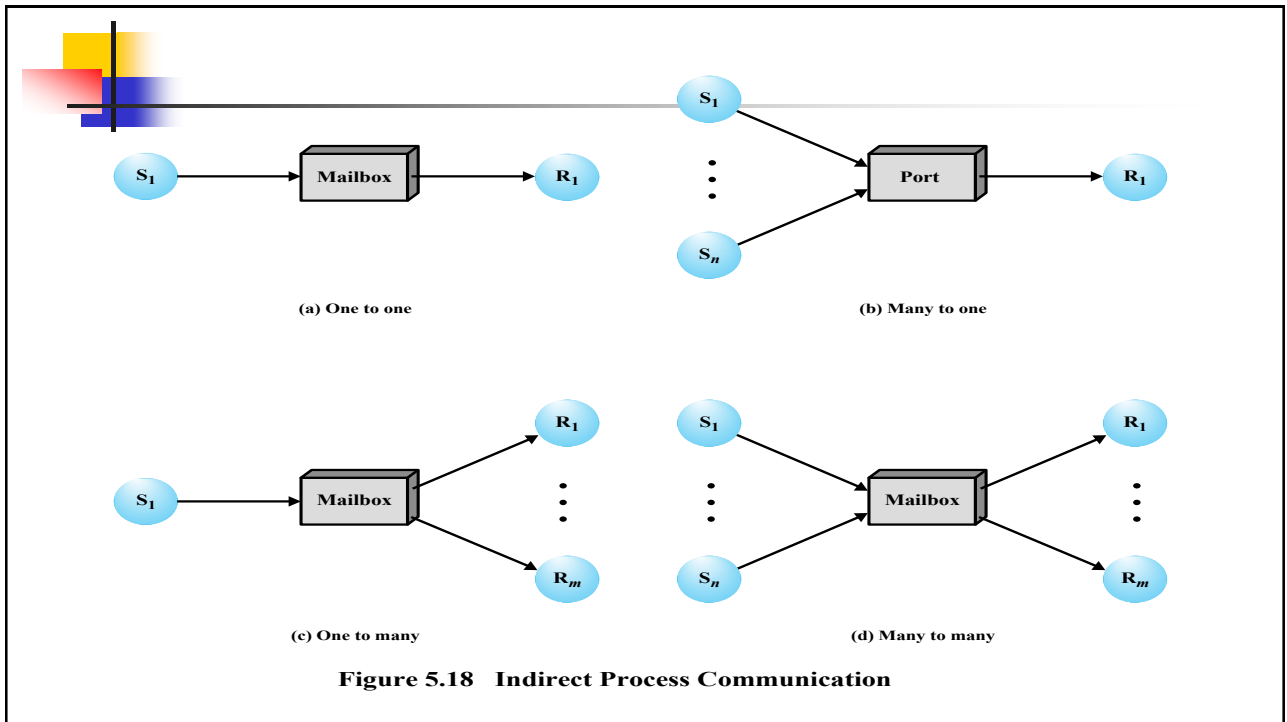


Figure 5.20 Mutual Exclusion Using Messages

```

/* program mutualexclusion */
const int n = /* number of processes */;
void P(int i)
{
    message msg;
    while (true) {
        receive (box, msg);
        /* critical section */
        send (box, msg);
        /* remainder */
    }
}
void main()
{
    create_mailbox (box);
    send (box, null);
    parbegin (P(1), P(2), . . ., P(n));
}

```

Figure 5.21
A Solution to the Bounded-Buffer Producer/Consumer
Problem Using Messages

```

const int
    capacity = /* buffering capacity */ ;
    null = /* empty message */ ;
int i;
void producer()
{
    message pmsg;
    while (true) {
        receive (mayproduce, pmsg);
        pmsg = produce();
        send (mayconsume, pmsg);
    }
}
void consumer()
{
    message cmsg;
    while (true) {
        receive (mayconsume, cmsg);
        consume (cmsg);
        send (mayproduce, null);
    }
}
void main()
{
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
    for (int i = 1; i <= capacity; i++) send (mayproduce,
null);
    parbegin (producer, consumer);
}

```

Readers/Writers Problem

- A data area is shared among many processes
 - some processes only read the data area, (readers) and some only write to the data area (writers)
- Conditions that must be satisfied:
 1. any number of readers may simultaneously read the file
 2. only one writer at a time may write to the file
 3. if a writer is writing to the file, no reader may read it

Figure 5.22
A Solution to the Readers/Writers Problem Using
Semaphores: Readers Have Priority

```

/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true) {
        semWait (x);
        readcount++;
        if (readcount == 1) semWait (wsem);
        semSignal (x);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer()
{
    while (true) {
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
    }
}
void main()
{
    readcount = 0;
    parbegin (reader, writer);
}

```

Table 5.6 State of the Process Queues for Program of Figure 5.23

Readers only in the system	• <i>wsem</i> set • no queues
Writers only in the system	• <i>wsem</i> and <i>rsem</i> set • writers queue on <i>wsem</i>
Both readers and writers with read first	• <i>wsem</i> set by reader • <i>rsem</i> set by writer • all writers queue on <i>wsem</i> • one reader queues on <i>rsem</i> • other readers queue on <i>z</i>
Both readers and writers with write first	• <i>wsem</i> set by writer • <i>rsem</i> set by writer • writers queue on <i>wsem</i> • one reader queues on <i>rsem</i> • other readers queue on <i>z</i>

A Solution to the Readers/Writers Problem Using Semaphores: Writers Have Priority

```

/* program readersandwriters */
int readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader()
{
    while (true) {
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1) semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0) semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true) {
        semWait (y);
        writecount++;
        if (writecount == 1) semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0) semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}

```



Figure 5.24 A Solution to the Readers/Writers Problem Using Message Passing

<pre> void reader(int i) { message rmsg; while (true) { rmsg = i; send (readrequest, rmsg); receive (mbox[i], rmsg); READUNIT (); rmsg = i; send (finished, rmsg); } } void writer(int j) { message rmsg; while (true) { rmsg = j; send (writerequest, rmsg); receive (mbox[j], rmsg); WRITEUNIT (); rmsg = j; send (finished, rmsg); } } </pre>	<pre> void controller() { while (true) { if (count > 0) { if (!empty (finished)) { receive (finished, msg); count++; } else if (!empty (writerequest)) { receive (writerequest, msg); writer id = msg.id; count = count - 100; } else if (!empty (readrequest)) { receive (readrequest, msg); count--; send (msg.id, "OK"); } } if (count == 0) { send (writer id, "OK"); receive (finished, msg); count = 100; } while (count < 0) { receive (finished, msg); count++; } } } </pre>
---	--



Summary

- Principles of concurrency
 - ✓ Race condition
 - ✓ OS concerns
 - ✓ Process interaction
 - ✓ Requirements for mutual exclusion
- Mutual exclusion: hardware support
 - ✓ Interrupt disabling
 - ✓ Special machine instructions
- Semaphores
 - ✓ Mutual exclusion
 - ✓ Producer/consumer problem
 - ✓ Implementation of semaphores
- Monitors
 - ✓ Monitor with signal
 - ✓ Alternate model of monitors with notify and broadcast
- Message passing
 - ✓ Synchronization
 - ✓ Addressing
 - ✓ Message format
 - ✓ Queueing discipline
 - ✓ Mutual exclusion
- Readers/writers problem
 - ✓ Readers have priority
 - ✓ Writers have priority